SmartHR AI - Code Walkthrough & Viva Preparation

A Step-by-Step Guide to Understanding the Project

Table of Contents

---

1. Project Overview in 2 Minutes

What is SmartHR AI? SmartHR AI is a web application that helps HR departments predict which employees might leave the company (attrition). You enter employee details like age, salary, satisfaction level, and the system tells you the risk of them leaving.

One-Line Summary: "It's a full-stack web app that uses employee data to predict attrition risk, with a Spring Boot backend, MySQL database, and Bootstrap frontend."

Key Numbers:

- 1,470 employee records (IBM dataset)
- 31 features per employee
- 7 frontend pages
- 5 REST API endpoints
- 3 database tables

Tech Stack (say this confidently):

- Backend: Spring Boot 3.5 + Java 17
- Database: MySQL 8.0
- Frontend: HTML5 + CSS3 + JavaScript + Bootstrap 5
- Charts: Chart.js
- Build tool: Maven

---

2. How the System Works (Big Picture)

```
USER (Browser)
      |
      | Opens login.html
      | Enters email + password
      v
  FRONTEND (JavaScript)
      |
      | Sends HTTP POST request
      | to http://localhost:8080/api/auth/login
      v
  BACKEND (Spring Boot)
      |
      | AuthController receives request
      | UserService validates credentials
      | UserRepository checks MySQL database
      v
  DATABASE (MySQL)
      |
      | Returns user data if credentials match
      v
  BACKEND returns JSON response
      |
      v
  FRONTEND stores user info in localStorage
      |
      | Redirects to dashboard.html
      v
  DASHBOARD loads
      |
      | Fetches /api/dashboard/stats
      | Renders charts with Chart.js
      v
  USER sees employee stats and charts
```

The same pattern repeats for every feature:

- Prediction: Frontend form -> POST /api/predictions/predict -> Backend saves to DB -> Returns result
- History: Page loads -> GET /api/predictions/history/{userId} -> Backend queries DB -> Returns list
- Reports: Page loads -> Fetches data -> Chart.js renders charts

3. File-by-File Walkthrough

3.1 Backend Entry Point

File: `SmarthrAiApplication.java`

```
@SpringBootApplication
  public class SmarthrAiApplication {
      public static void main(String[] args) {
          SpringApplication.run(SmarthrAiApplication.class, args);
      }
  }
```

What it does: This is the main class that starts the entire Spring Boot application. When you run this, Spring:

1. Scans all packages for components (controllers, services, etc.)
2. Sets up an embedded Tomcat server on port 8080
3. Connects to MySQL using application.properties config
4. Auto-creates/updates database tables from entity classes

Viva Q: "How does Spring Boot start?" **A:** "The `@SpringBootApplication` annotation enables auto-configuration, component scanning, and configuration. `SpringApplication.run()` bootstraps the application, starts the embedded Tomcat server, and initializes all Spring beans."

3.2 Entities (Data Models)

`User.java` - Represents a registered user

```
@Entity
  @Table(name = "users")
  public class User {
      @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
      private Long id;
      private String email;
      private String password;
      private String fullName;
      private String phone;
      private String role;
      private boolean active;
      private LocalDateTime createdAt;
      private LocalDateTime updatedAt;
  }
```

What it does: Maps to the `users` table in MySQL. Each field becomes a column. JPA/Hibernate automatically creates this table.

Key annotations:

- `@Entity` - Tells JPA this is a database table
- `@Table(name = "users")` - Table name in MySQL
- `@Id` + `@GeneratedValue` - Auto-increment primary key
- `@PrePersist` - Runs before inserting (sets timestamps)

`Employee.java` - Represents an employee from the dataset

```
@Entity
  @Table(name = "employees")
  public class Employee {
      private Integer age;
      private String attrition;
      private String department;
      private Double monthlyIncome;
      private String overtime;
      // ... 31 fields total
  }
```

What it does: Maps to the `employees` table containing 1,470 records from the IBM HR Analytics dataset. This data is used for dashboard statistics.

`PredictionHistory.java` - Stores prediction results

```
@Entity
  @Table(name = "prediction_history")
  public class PredictionHistory {
      @Id private Long id;
      @ManyToOne(fetch = FetchType.LAZY)
      @JoinColumn(name = "user_id")
      private User user;
      private Integer age;
      private String department;
      private String attritionPrediction;
      private Double confidenceScore;
      private LocalDateTime predictedAt;
  }
```

What it does: Stores every prediction made by users. The `@ManyToOne` creates a foreign key to the `users` table.

Viva Q: "What is the relationship between User and PredictionHistory?" **A:** "It's a one-to-many relationship. One user can make many predictions. The `@ManyToOne` annotation on PredictionHistory creates a foreign key `user_id` pointing to the `users` table."

3.3 Repositories (Data Access)

```
public interface UserRepository extends JpaRepository<User, Long> {
      Optional<User> findByEmail(String email);
      boolean existsByEmail(String email);
  }

  public interface EmployeeRepository extends JpaRepository<Employee, Long> {
      @Query("SELECT COUNT(e) FROM Employee e WHERE e.attrition = 'Yes'")
      long countByAttritionYes();
  }

  public interface PredictionHistoryRepository extends JpaRepository<PredictionHistory,
  Long> {
      List<PredictionHistory> findByUserIdOrderByPredictedAtDesc(Long userId);
  }
```

What they do: Each repository is an interface that extends `JpaRepository`. Spring Data JPA automatically generates the implementation. You just declare method names and Spring creates the SQL queries.

How Spring creates queries from method names:

- `findByEmail(String email)` -> `SELECT * FROM users WHERE email = ?`
- `existsByEmail(String email)` -> `SELECT COUNT(*) FROM users WHERE email = ? > 0`
- `findByUserIdOrderByPredictedAtDesc(Long userId)` -> `SELECT * FROM prediction_history WHERE user_id = ? ORDER BY predicted_at DESC`

Viva Q: "How does Spring Data JPA generate queries?" **A:** "Spring Data JPA uses method name parsing. It parses the method name like `findByEmail` and automatically generates the corresponding JPQL query. For complex queries, we use the `@Query` annotation with custom JPQL."

3.4 Services (Business Logic)

`UserService.java`

```
@Service
  public class UserService {
      private final UserRepository userRepository;

      public AuthResponse register(RegisterRequest request) {
          if (userRepository.existsByEmail(request.getEmail())) {
              throw new BadRequestException("Email already registered");
          }
          User user = User.builder()
              .fullName(request.getFullName())
              .email(request.getEmail())
              .password(request.getPassword())
              .role("USER")
              .active(true)
              .build();
          userRepository.save(user);
          return AuthResponse.builder()
              .userId(user.getId())
              .token("simple-token-" + user.getId())
              .build();
      }
  }
```

What it does: Contains the business logic for user operations. The controller calls the service, and the service calls the repository.

Viva Q: "Why do we need a Service layer?" **A:** "The Service layer separates business logic from HTTP concerns (Controller) and data access (Repository). This follows the Single Responsibility Principle and makes the code testable and maintainable."

`PredictionService.java` (Most Important!)

```
public PredictionResponse predict(PredictionRequest request, Long userId) {
      // 1. Find the user
      User user = userRepository.findById(userId).orElseThrow(...);

      // 2. Run the prediction algorithm
      String prediction = simulatePrediction(request);
      double confidence = 0.6 + Math.random() * 0.35;

      // 3. Save to database
      PredictionHistory history = PredictionHistory.builder()
          .user(user).age(request.getAge())
          .attritionPrediction(prediction)
          .confidenceScore(confidence)
          .build();
      predictionRepository.save(history);

      // 4. Return result
      return PredictionResponse.builder()
          .id(history.getId())
          .attritionPrediction(prediction)
          .confidenceScore(history.getConfidenceScore())
          .build();
  }
```

What it does: This is the core of the project. It:

1. Looks up the user who made the prediction
2. Runs the rule-based prediction algorithm
3. Saves the prediction to the database
4. Returns the result to the frontend

3.5 Controllers (API Layer)

```
@RestController
  @RequestMapping("/api/predictions")
  public class PredictionController {

      @PostMapping("/predict")
      public ResponseEntity<ApiResponse<PredictionResponse>> predict(
              @RequestBody PredictionRequest request,
              @RequestParam(defaultValue = "1") Long userId) {
          PredictionResponse response = predictionService.predict(request, userId);
          return ResponseEntity.ok(ApiResponse.success("Prediction completed", respons
  e));
      }

      @GetMapping("/history/{userId}")
      public ResponseEntity<ApiResponse<List<PredictionResponse>>> getHistory(
              @PathVariable Long userId) {
          List<PredictionResponse> history = predictionService.getHistory(userId);
          return ResponseEntity.ok(ApiResponse.success(history));
      }
  }
```

What it does: Receives HTTP requests, calls the service layer, and returns JSON responses.

Key annotations:

- `@RestController` - Marks as REST API controller
- `@RequestMapping("/api/predictions")` - Base URL path
- `@PostMapping("/predict")` - Handles POST requests
- `@RequestBody` - Reads JSON body from request
- `@RequestParam` - Reads URL query parameter
- `@PathVariable` - Reads URL path variable

Viva Q: "What's the difference between @RequestParam and @PathVariable?" **A:** "@RequestParam reads query parameters like `?userId=1`, while @PathVariable reads URL path segments like `/history/1`. Query params are optional with defaults; path variables are part of the URL structure."

3.6 DTOs (Data Transfer Objects)

Why DTOs? We don't send database entities directly to the frontend. DTOs control what data is exposed.

```
// Request DTO - What the frontend sends
  public class PredictionRequest {
      private Integer age;
      private String department;
      private Double monthlyIncome;
      // ... 24 fields
  }

  // Response DTO - What the backend returns
  public class PredictionResponse {
      private Long id;
      private String attritionPrediction;
      private Double confidenceScore;
      private LocalDateTime predictedAt;
  }

  // Generic API wrapper
  public class ApiResponse<T> {
      private boolean success;
      private String message;
      private T data;
  }
```

Viva Q: "Why use DTOs instead of returning entities directly?" **A:** "DTOs provide several benefits: 1) Security - we don't expose sensitive fields like passwords, 2) Flexibility - we can reshape data for the frontend, 3) Decoupling - frontend doesn't depend on database schema, 4) Performance - we can send only needed fields."

3.7 Exception Handling

```
@RestControllerAdvice
  public class GlobalExceptionHandler {

      @ExceptionHandler(ResourceNotFoundException.class)
      public ResponseEntity<ApiResponse<Void>> handleNotFound(ResourceNotFoundException
  ex) {
          return ResponseEntity.status(HttpStatus.NOT_FOUND)
              .body(ApiResponse.error(ex.getMessage()));
      }

      @ExceptionHandler(BadRequestException.class)
      public ResponseEntity<ApiResponse<Void>> handleBadRequest(BadRequestException ex)
  {
          return ResponseEntity.status(HttpStatus.BAD_REQUEST)
              .body(ApiResponse.error(ex.getMessage()));
      }

      @ExceptionHandler(MethodArgumentNotValidException.class)
      public ResponseEntity<ApiResponse<Map<String, String>>> handleValidation(...) {
          // Returns field-level validation errors
      }
  }
```

What it does: Catches all exceptions globally and returns consistent JSON error responses. This prevents stack traces from being sent to the frontend.

Viva Q: "What is @RestControllerAdvice?" **A:** "It's a specialization of @ControllerAdvice that handles exceptions across all controllers. @ExceptionHandler methods inside it catch specific exception types and return appropriate HTTP responses, providing centralized error handling."

3.8 CORS Configuration

```
@Configuration
  public class CorsConfig {
      @Bean
      public CorsFilter corsFilter() {
          CorsConfiguration config = new CorsConfiguration();
          config.setAllowedOrigins(List.of(
              "http://localhost:3000",
              "http://localhost:5500",
              "file://"
          ));
          config.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE"));
          config.setAllowedHeaders(List.of("*"));
          config.setAllowCredentials(true);
          // ...
      }
  }
```

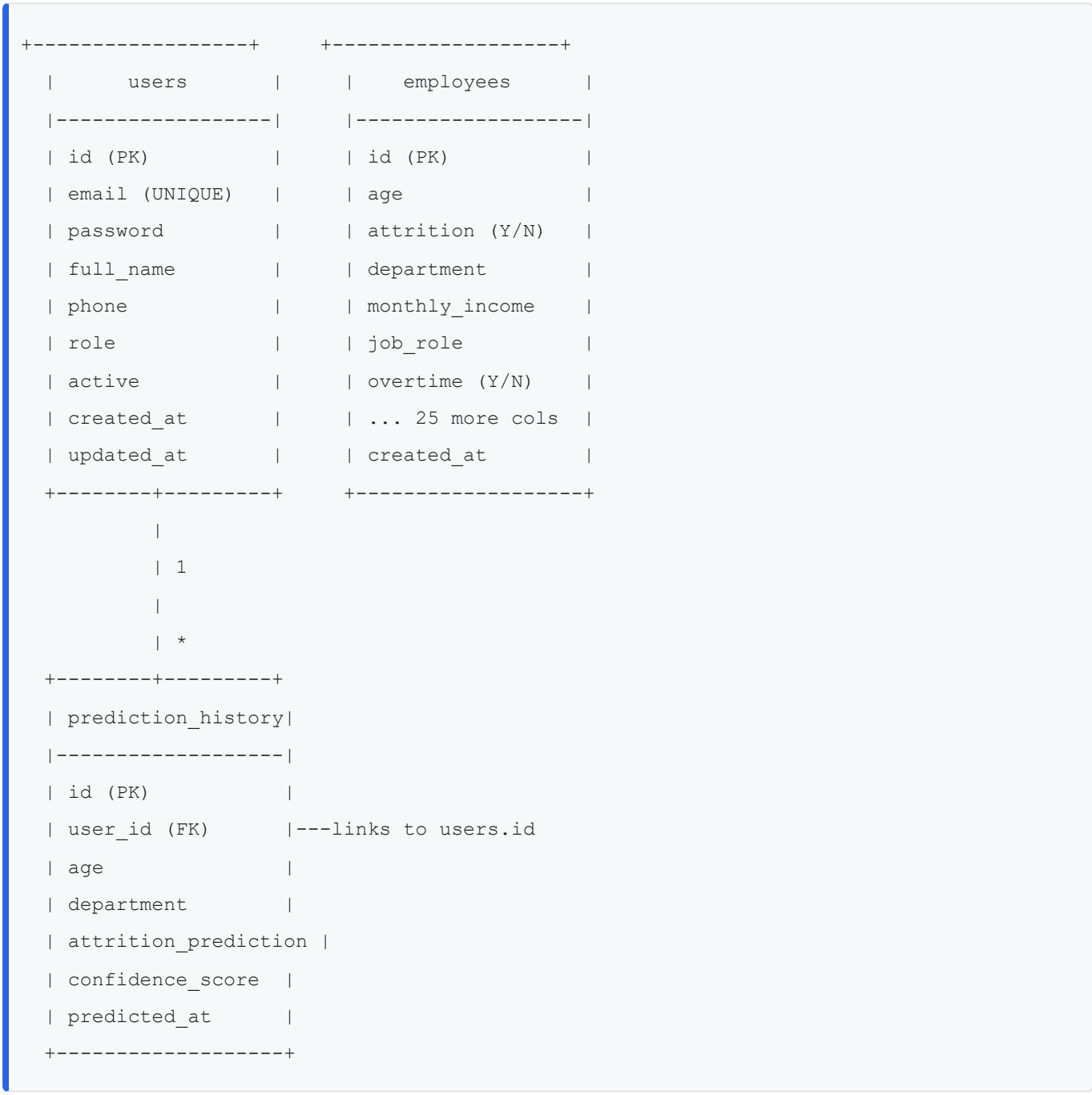
What it does: Allows the frontend (running on a different port/protocol) to make API calls to the backend. Without CORS config, browsers block cross-origin requests.

Viva Q: "What is CORS and why do we need it?" **A:** "CORS (Cross-Origin Resource Sharing) is a browser security mechanism. By default, browsers block requests from one origin (e.g., file://) to another (e.g., localhost:8080). The CORS filter on the backend tells the browser to allow these requests."

---

4. Database Explained

Table Overview



Key Points:

- `employees` table has 1,470 rows (IBM dataset) - never modified by users
- `users` table grows as people register
- `prediction_history` grows each time someone makes a prediction
- `user_id` in prediction_history is a foreign key to `users.id`

SQL Queries You Should Know

```
-- Count employees by attrition status
  SELECT COUNT(*) FROM employees WHERE attrition = 'Yes';  -- Returns 237

  -- Get user by email (for login)
  SELECT * FROM users WHERE email = 'test@test.com';

  -- Get prediction history for a user
  SELECT * FROM prediction_history WHERE user_id = 1 ORDER BY predicted_at DESC;

  -- Dashboard statistics
  SELECT COUNT(*) as total FROM employees;
  SELECT COUNT(*) as attrition FROM employees WHERE attrition = 'Yes';
```

---

5. API Endpoints Explained

POST /api/auth/register

Request:

```
{
      "fullName": "Govind",
      "email": "govind@test.com",
      "phone": "9876543210",
      "password": "123456"
  }
```

Response (200 OK):

```
{
      "success": true,
      "message": "Registration successful",
      "data": {
          "userId": 3,
          "fullName": "Govind",
          "email": "govind@test.com",
          "role": "USER",
          "token": "simple-token-3"
      }
  }
```


Flow: Frontend sends form data -> AuthController -> UserService.register() -> Checks email uniqueness -> Saves to DB -> Returns token

---

POST /api/auth/login

Request:


```
{
      "email": "govind@test.com",
      "password": "123456"
  }
```


Response (200 OK):


```
{
      "success": true,
      "message": "Login successful",
      "data": {
          "userId": 3,
          "fullName": "Govind",
          "email": "govind@test.com",
          "role": "USER",
          "token": "simple-token-3"
      }
  }
```


Flow: Frontend sends credentials -> AuthController -> UserService.login() -> Validates password -> Returns user data + token

GET /api/dashboard/stats

Response (200 OK):

```
{
      "success": true,
      "message": "Success",
      "data": {
          "totalEmployees": 1470,
          "activeEmployees": 2,
          "attritionCount": 237,
          "totalPredictions": 5,
          "attritionRate": 16.1
      }
  }
```

Flow: Dashboard loads -> dashboard.js calls fetch() -> DashboardController -> DashboardService -> Counts from 3 tables -> Returns aggregated stats

---

POST /api/predictions/predict?userId=3

Request:

```
{
      "age": 28,
      "department": "Sales",
      "jobRole": "Sales Executive",
      "monthlyIncome": 4500,
      "overtime": "Yes",
      "jobSatisfaction": "Low",
      "environmentSatisfaction": "Medium",
      "workLifeBalance": "Bad",
      "totalWorkingYears": 3,
      "yearsAtCompany": 1,
      "distanceFromHome": 20,
      "maritalStatus": "Single",
      "gender": "Male"
  }
```

Response (200 OK):

```
{
      "success": true,
      "message": "Prediction completed",
      "data": {
          "id": 4,
          "attritionPrediction": "Yes",
          "confidenceScore": 0.82,
          "predictedAt": "2026-07-19T10:30:00"
      }
  }
```

Flow: User fills form -> prediction.js sends POST -> PredictionController -> PredictionService.predict() -> simulatePrediction() calculates risk score -> Saves to prediction_history -> Returns result

---

GET /api/predictions/history/3

Response (200 OK):

```
{
      "success": true,
      "message": "Success",
      "data": [
          {
              "id": 4,
              "attritionPrediction": "Yes",
              "confidenceScore": 0.82,
              "predictedAt": "2026-07-19T10:30:00"
          },
          {
              "id": 2,
              "attritionPrediction": "No",
              "confidenceScore": 0.71,
              "predictedAt": "2026-07-19T09:15:00"
          }
      ]
  }
```

Flow: history.js calls fetch() -> PredictionController -> PredictionService.getHistory() -> Queries prediction*history table WHERE user*id = 3 -> Returns sorted list

6. Frontend Explained

6.1 Page Architecture

Every page follows this structure:

```
<body>
      <div class="wrapper d-flex">

          <aside class="sidebar">
              <div class="logo">SmartHR AI</div>
              <nav>
                  <a href="dashboard.html">Dashboard</a>
                  <a href="prediction.html">Prediction</a>
                  <a href="history.html">History</a>
                  <a href="reports.html">Reports</a>
                  <a href="profile.html">Profile</a>
              </nav>
              <button>Logout</button>
          </aside>


          <main class="main-content">

          </main>
      </div>


      <script src="js/init-theme.js"></script>
      <script src="js/page-specific.js"></script>
  </body>
```

6.2 How Frontend Talks to Backend

```
// Example: Making a prediction
  async function predictAttrition() {
      const response = await fetch(
          "http://localhost:8080/api/predictions/predict?userId=" + user.userId,
          {
              method: "POST",
              headers: { "Content-Type": "application/json" },
              body: JSON.stringify({
                  age: 28,
                  department: "Sales",
                  overtime: "Yes",
                  // ... other fields
              })
          }
      );
      const result = await response.json();
      // result.data.attritionPrediction = "Yes"
      // result.data.confidenceScore = 0.82
  }
```

6.3 State Management with localStorage

The frontend uses `localStorage` (browser storage) to persist data:

```
// After login, store user data
  localStorage.setItem("user", JSON.stringify({
      userId: 3,
      fullName: "Govind",
      email: "govind@test.com",
      token: "simple-token-3"
  }));

  // On any page, retrieve user data
  const user = JSON.parse(localStorage.getItem("user") || "{}");
  const name = user.fullName; // "Govind"

  // Settings are also stored
  localStorage.setItem("smarthr_settings", JSON.stringify({
      theme: "dark",
      sidebar: "right",
      primaryColor: "#2563eb"
  }));
```

6.4 Dark Mode Implementation

Step 1: `init-theme.js` runs on every page load:

```
(function() {
      var settings = JSON.parse(localStorage.getItem("smarthr_settings") || "{}");
      if (settings.theme === "dark") {
          document.body.classList.add("dark-theme");
      }
  })();
```

Step 2: CSS rules activate when `dark-theme` class is present:

```
body.dark-theme {
      background: #0f172a;
      color: #e2e8f0;
  }
  body.dark-theme .sidebar {
      background: #0f172a;
  }
  body.dark-theme .card {
      background: #1e293b;
  }
```

Step 3: User changes theme in Settings -> `applySettings()` toggles the class:

```
function applyDarkTheme() {
      document.body.classList.add("dark-theme");
  }
  function applyLightTheme() {
      document.body.classList.remove("dark-theme");
  }
```

---

7. Prediction Algorithm Deep Dive

The Algorithm Step-by-Step

```
INPUT: Employee attributes (age, overtime, income, etc.)
  OUTPUT: "Yes" (will leave) or "No" (will stay)

  PROCESS:
  1. Start with riskScore = 0
  2. Check each factor:
     - Overtime = "Yes"?     -> riskScore += 3  (BIGGEST factor)
     - Age < 30?             -> riskScore += 2
     - Income < 5000?        -> riskScore += 2
     - Distance > 15 miles?  -> riskScore += 2
     - Job Satisfaction Low? -> riskScore += 2
     - Env Satisfaction Low? -> riskScore += 2
     - Work-Life Bad?        -> riskScore += 2
     - Working Years < 3?    -> riskScore += 1
     - Years at Company < 2? -> riskScore += 1
     - Single?               -> riskScore += 1
     - Job Sat Very High?    -> riskScore -= 1  (protective)
     - Env Sat Very High?    -> riskScore -= 1  (protective)
     - Work-Life Good?       -> riskScore -= 1  (protective)
  3. riskScore = max(0, riskScore)  (can't go negative)
  4. If riskScore >= 5: return "Yes"
     Else: return "No"
```

Example Calculations

Example 1: High Risk Employee

```
Age: 25 (< 30 -> +2)
  Overtime: Yes (+3)
  Income: 4000 (< 5000 -> +2)
  Job Satisfaction: Low (+2)
  Total riskScore: 9 -> "Yes" (Attrition)
```

Example 2: Low Risk Employee

```
Age: 40
  Overtime: No
  Income: 12000
  Job Satisfaction: Very High (-1)
  Work-Life Balance: Best (-1)
  Total riskScore: -2 -> max(0, -2) = 0 -> "No" (Retention)
```

Example 3: Borderline Employee

```
Age: 28 (+2)
  Overtime: No
  Income: 4500 (+2)
  Distance: 18 (+2)
  Job Satisfaction: Medium
  Total riskScore: 6 -> "Yes" (Attrition)
```

---

8. Authentication Flow

Complete Login Flow

```
1. User opens login.html
  2. Enters email: govind@test.com, password: 123456
  3. Clicks "Login" button

  4. login.js captures form submit event:
     event.preventDefault()

  5. login.js sends POST request:
     fetch("http://localhost:8080/api/auth/login", {
         method: "POST",
         headers: { "Content-Type": "application/json" },
         body: JSON.stringify({
             email: "govind@test.com",
             password: "123456"
         })
     })

  6. Backend receives request:
     AuthController.login() -> UserService.login()
     -> UserRepository.findByEmail("govind@test.com")
     -> Checks password matches
     -> Returns AuthResponse with token

  7. Frontend receives response:
     if (result.success) {
         localStorage.setItem("user", JSON.stringify(result.data));
         window.location.href = "dashboard.html";
     }

  8. Dashboard loads:
     dashboard.js reads localStorage for user data
     Displays "Welcome back, Govind" in the navbar
     Fetches /api/dashboard/stats for statistics
```

Complete Prediction Flow

```
1. User navigates to prediction.html
  2. Fills form: Age=28, Department=Sales, Overtime=Yes, etc.
  3. Clicks "Predict" button

  4. prediction.js captures form submit:
     const data = {
         age: 28,
         department: "Sales",
         overtime: "Yes",
         monthlyIncome: 4500,
         // ... all form fields
     };

  5. Sends POST request:
     fetch("http://localhost:8080/api/predictions/predict?userId=" + user.userId, {
         method: "POST",
         body: JSON.stringify(data)
     })

  6. Backend processes:
     PredictionService.predict()
     -> simulatePrediction(request) calculates riskScore = 9
     -> riskScore >= 5, so prediction = "Yes"
     -> Creates PredictionHistory entity
     -> predictionRepository.save() saves to MySQL
     -> Returns PredictionResponse

  7. Frontend displays result:
     Shows "Yes - Attrition Risk" with red badge
     Shows confidence: 82%
     Shows timestamp
```

9. Data Flow Diagrams

Context Diagram (Level 0)

```
                  +-----------+
    Register/       |           |      Dashboard Stats
    Login      --->|           |--->
                    |           |
    Prediction  --->| SmartHR AI|      Prediction
    Data            |           |      Result
                    |           |
    View History --->|           |--->
                    |           |      History
                    +-----+-----+
                          |
                     MySQL DB
```

Level 1 Data Flow

```
[User] --credentials--> [AuthController] --calls--> [UserService] --queries--> [MySQL]
  [User] --prediction----> [PredictionController] --calls--> [PredictionService] --save
  s--> [MySQL]
  [User] --view----------> [DashboardController] --calls--> [DashboardService] --querie
  s--> [MySQL]
```

---

10. Common Viva Questions & Answers

Q1: What is your project about?

A: "SmartHR AI is a full-stack web application that predicts employee attrition. It uses the IBM HR Analytics dataset of 1,470 employees and evaluates factors like overtime, age, income, and job satisfaction to predict whether an employee is likely to leave the company."

Q2: What technologies did you use and why?

A:

- "Spring Boot for the backend because it simplifies REST API development with auto-configuration and JPA integration."
- "MySQL for the database because it's reliable, free, and well-suited for relational data."
- "Bootstrap 5 for the frontend because it provides responsive components and speeds up UI development."
- "Chart.js for data visualization because it's lightweight and easy to integrate."

Q3: How does the prediction algorithm work?

A: "It's a rule-based scoring algorithm. Each employee attribute contributes a weighted risk score. For example, overtime adds +3 points (highest weight), age under 30 adds +2, low income adds +2. If the total risk score reaches 5 or above, the system predicts 'Yes' (attrition likely). The weights are based on feature importance analysis from the IBM HR Analytics dataset."

Q4: What is Spring Boot?

A: "Spring Boot is a Java framework that simplifies building production-ready applications. It provides auto-configuration, embedded servers (Tomcat), and Spring Data JPA for database operations. It eliminates much of the boilerplate code needed in traditional Spring applications."

Q5: What is JPA/Hibernate?

A: "JPA (Java Persistence API) is a specification for mapping Java objects to database tables. Hibernate is the most popular implementation. We use Spring Data JPA which provides repository interfaces - we define method names like findByEmail() and Spring automatically generates the SQL queries."

Q6: What is the MVC pattern?

A: "MVC stands for Model-View-Controller. In our project: Models are the Entity classes (User, Employee, PredictionHistory), Views are the HTML pages, and Controllers are the REST API controllers. This separation makes the code organized and maintainable."

Q7: What is CORS?

A: "CORS (Cross-Origin Resource Sharing) is a security mechanism in browsers. Since our frontend and backend run on different origins (different ports/protocols), browsers would block requests by default. The CorsConfig in our backend tells the browser to allow requests from our frontend's origin."

Q8: What is the difference between GET and POST?

A: "GET requests retrieve data (like fetching dashboard stats or prediction history). They don't modify any data. POST requests send data to the server to create or process something (like registering a user or making a prediction). POST sends data in the request body; GET sends parameters in the URL."

Q9: What is a DTO?

A: "DTO (Data Transfer Object) is a design pattern. Instead of sending database entities directly to the frontend, we use DTOs to control what data is exposed. For example, our User entity has a password field, but the AuthResponse DTO only includes userId, name, email, role, and token - never the password."

Q10: How is the dataset imported?

A: "I wrote a Python script (import_dataset.py) that reads the IBM HR Analytics CSV file using the csv module and inserts each row into the MySQL employees table using mysql-connector-python. It imported all 1,470 rows with 31 columns."

Q11: What is @SpringBootApplication?

A: "It's a convenience annotation that combines three annotations: @Configuration (marks the class as a source of bean definitions), @EnableAutoConfiguration (enables Spring Boot's auto-configuration), and @ComponentScan (scans the package for components like controllers and services)."

Q12: What is @RestControllerAdvice?

A: "It's a global exception handler. When any controller throws an exception, the @ExceptionHandler methods in this class catch it and return a proper JSON response instead of a stack trace. We handle 404 (not found), 400 (bad request), validation errors, and general exceptions."

Q13: How does the dark mode work?

A: "When the user enables dark mode in Settings, a 'dark-theme' CSS class is added to the body element. All dark mode styles use 'body.dark-theme' as the selector. The preference is saved in localStorage so it persists across page loads and navigation. A small init-theme.js file runs on every page to check localStorage and apply the class before the page renders."

Q14: Why did you choose a rule-based algorithm over ML?

A: "The current version uses a rule-based algorithm for simplicity and interpretability. Every prediction can be explained by specific factors. However, the project includes an `/ml/` directory with Python scripts for training Random Forest and other ML models, which can replace the rule-based algorithm in future versions."

Q15: What is the significance of the IBM HR Analytics dataset?

A: "It's a widely-used benchmark dataset in HR analytics with 1,470 employee records and 31 features. It contains realistic employee attributes like age, department, income, satisfaction levels, and an attrition column (Yes/No) that serves as the target variable. It's ideal for building and testing prediction models."

Q16: How would you improve this project?

A: "Several improvements: 1) Replace rule-based algorithm with trained ML models, 2) Implement JWT authentication with password hashing (BCrypt), 3) Add real-time updates with WebSockets, 4) Add export to PDF/Excel, 5) Implement role-based access control (admin vs regular user), 6) Add email notifications for high-risk alerts."

Q17: What design patterns did you use?

A: "1) MVC (Model-View-Controller) - separating frontend, API, and data layers. 2) Repository Pattern - abstracting database operations behind interfaces. 3) DTO Pattern - controlling data transfer between layers. 4) Builder Pattern (via Lombok) - for clean object construction. 5) Global Exception Handling - centralized error handling."

Q18: What is Lombok?

A: "Lombok is a Java library that reduces boilerplate code. Instead of writing getters, setters, constructors, and builder patterns manually, we use annotations like @Getter, @Setter, @Builder, @NoArgsConstructor, @AllArgsConstructor. It generates these methods at compile time."

Q19: How is the password stored?

A: "In this academic project, passwords are stored as plain text in the database for simplicity. In a production system, I would use BCryptPasswordEncoder to hash passwords before storage, and compare hashes during login instead of plain text comparison."

Q20: What is the role of application.properties?

A: "It's Spring Boot's configuration file. It contains: server port (8080), MySQL connection details (URL, username, password), JPA/Hibernate settings (ddl-auto=update for auto table creation, show-sql for debugging), and JSON formatting options."

---

11. Key Terms Glossary

Term	Definition
Attrition	Employee leaving the company voluntarily
REST API	Architectural style for web services using HTTP methods
JSON	JavaScript Object Notation - data format for API communication
JPA	Java Persistence API - specification for database mapping
Hibernate	Implementation of JPA for ORM (Object-Relational Mapping)
ORM	Mapping Java objects to database tables automatically
CORS	Cross-Origin Resource Sharing - browser security mechanism
DTO	Data Transfer Object - controls data sent between layers
MVC	Model-View-Controller - software architecture pattern
Lombok	Java library to reduce boilerplate code
JpaRepository	Spring Data interface for database CRUD operations
@SpringBootApplication	Main annotation to start a Spring Boot application
@RestController	Marks a class as REST API controller
@Entity	Marks a class as a database table mapping
@Service	Marks a class as a business logic component
@Repository	Marks an interface as a data access component
DDL	Data Definition Language (CREATE, ALTER, DROP tables)
JPQL	Java Persistence Query Language (database queries in Java)
Embedded Tomcat	Web server included inside the Spring Boot application
Auto-Configuration	Spring Boot automatically configures beans based on dependencies
localStorage	Browser storage that persists data across sessions
Fetch API	Modern JavaScript API for making HTTP requests
Bootstrap	CSS framework for responsive web design
Chart.js	JavaScript library for creating charts and graphs

12. How to Explain Your Project in 5 Minutes

Minute 1: Problem & Solution

"Employee attrition costs companies 50-200% of an employee's salary per departure. My project, SmartHR AI, is a web application that helps HR departments predict which employees are likely to leave, so they can take proactive retention measures."

Minute 2: Tech Stack

"I built it with Spring Boot and Java for the backend, MySQL for the database, and HTML/CSS/JavaScript with Bootstrap for the frontend. The dataset is IBM's HR Analytics dataset with 1,470 employee records."

Minute 3: Key Features

"The system has 7 pages: login, register, dashboard with real-time statistics and charts, a prediction form that evaluates 24 employee attributes, prediction history tracking, analytics reports with Chart.js, and a profile page. It also has a dark mode theme."

Minute 4: How Prediction Works

"The prediction engine uses a weighted scoring algorithm. Factors like overtime (+3 points), age under 30 (+2), and low income (+2) increase risk. Protective factors like high job satisfaction reduce the score. If the total risk score reaches 5 or above, it predicts attrition. Each prediction is saved to the database with a confidence score."

Minute 5: Results & Learning

"The system correctly aggregates statistics from 1,470 records, predicts attrition based on multiple factors, and stores all predictions. I learned full-stack development, REST API design, database modeling, and how to apply ML concepts to real-world problems. In the future, I plan to integrate actual ML models from the /ml/ directory."

Good luck with your viva, Govind!